Week-7 (DAY-31) Hands-On – Keerthivasan R

Problem 1: Contact Management Web Application (Using Dependency Injection)

Scenario:

ABC Infotech wants to improve their application design by following **best practices like Dependency Injection (DI)**. Instead of handling all logic inside the controller, they want to introduce a **service layer** to manage contact operations.

The application should still:

- Add contacts
- View all contacts
- Search contact by ID

But now using **DI for better architecture**.

Requirements:

1. Model Class

Create ContactInfo class with auto-implemented properties:

Name	Type	Modifier
ContactId	int	public
FirstName	string	public
LastName	string	public
CompanyName	string	public
EmailId	string	public
MobileNo	long	public
Designation	string	public

2. Create Service Layer (DI Implementation)

Create:

- Interface → IContactService

Include methods:

- List<ContactInfo> GetAllContacts()
- ContactInfo GetContactById(int id)
- void AddContact(ContactInfo contact)

- Implementation → ContactService

- Maintain:

private static List<ContactInfo> contacts = new List<ContactInfo>();

Implement all methods:

- Add contact to list
- Return all contacts
- Search contact by ID

3.Register Service in DI Container

In Program.cs:

builder.Services.AddSingleton<IContactService, ContactService>();

4.Controller: ContactController

Inject service using **constructor injection**:

5. Action Methods

Function	**Description**
ShowContacts()	Use _contactService.GetAllContacts() and display list in tabular format
GetContactById(int id)	Use _contactService.GetContactById(id) and display contact
AddContact()	Return view to accept contact details
[HttpPost] AddContact(ContactInfo contactInfo)	Call _contactService.AddContact(contactInfo) and redirect to ShowContacts

6. Startup Configuration

Set default route to:

- Contact → ShowContacts

Technical Constraints

- MUST use:
    - Dependency Injection
    - Interface + Service class
    - Constructor Injection
    - Attribute-based routing (optional but recommended)

- Do NOT use:
    - Database
    - Static List inside Controller ❌
    - Business logic inside Controller ❌
- Static List allowed ONLY inside Service class

Expectations

1. Proper DI implementation
2. Controller should ONLY:
    - Receive request
    - Call service
    - Return response
3. Service should:
    - Handle all business logic
    - Manage data
4. Clean separation of:
    - Controller vs Service

Learning Outcome

After completing this problem, students will:

- Understand Dependency Injection in ASP.NET Core MVC
- Learn:
    - Interface-based design
    - Service layer implementation
    - Constructor injection
- Realize:
    - Why fat controllers are bad
    - How DI improves maintainability & testability

CODE:

📁 Model / `ContactInfo.cs`

```
namespace ContactManagementApp.Models
{
    public class ContactInfo
    {
        public int ContactId { get; set; }
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string CompanyName { get; set; } = string.Empty;
        public string EmailId { get; set; } = string.Empty;
        public long MobileNo { get; set; }
        public string Designation { get; set; } = string.Empty;
    }
}
```

📁 Service/ `IContactService.cs`

```
using ContactManagementApp.Models;

namespace ContactManagementApp.Services
{
    public interface IContactService
    {
        List<ContactInfo> GetAllContacts();
        ContactInfo? GetContactById(int id);
        void AddContact(ContactInfo contact);
    }
}
```

📁 Service/ `ContactService.cs`

```
using ContactManagementApp.Models;

namespace ContactManagementApp.Services
{
    public class ContactService : IContactService
    {
        // 'static' ensures the data doesn't reset on every page load
        private static List<ContactInfo> contacts = new List<ContactInfo>();

        public List<ContactInfo> GetAllContacts()
        {
            return contacts;
        }

        public ContactInfo? GetContactById(int id)
        {
            return contacts.FirstOrDefault(c => c.ContactId == id);
        }

        public void AddContact(ContactInfo contact)
        {
            // Auto-generate ID if it's 0
            if (contact.ContactId == 0)
            {
                contact.ContactId = contacts.Count > 0 ? contacts.Max(c => c.ContactId) + 1 : 1;
            }
            contacts.Add(contact);
        }
    }
}
```

📁 Controller/ `ContactController.cs`

```
using Microsoft.AspNetCore.Mvc;
using ContactManagementApp.Models;
using ContactManagementApp.Services;
```

```
namespace ContactManagementApp.Controllers{
    public class ContactController : Controller    {
        private readonly IContactService _contactService;
        // Constructor Injection
        public ContactController(IContactService contactService)        {
            _contactService = contactService;
        }
        // Display all contacts
        public IActionResult ShowContacts()        {
            var contacts = _contactService.GetAllContacts();
            return View(contacts);
        }
        // Search contact by ID
        public IActionResult GetContactById(int id)        {
            var contact = _contactService.GetContactById(id);
            if (contact == null)
            {
                ViewBag.Message = "Contact not found!";
                return View("ContactNotFound");
            }
            return View(contact);
        }
        // GET: Show Add Contact form
        public IActionResult AddContact()        {
            return View();
        }
        // POST: Add new contact
        [HttpPost]
        public IActionResult AddContact(ContactInfo contactInfo)        {
            if (ModelState.IsValid)
            {
                _contactService.AddContact(contactInfo);
                return RedirectToAction("ShowContacts");
            }
            return View(contactInfo);        }    }}
```

📁 Program.cs

```
using ContactManagementApp.Services;

var builder = WebApplication.CreateBuilder(args);

// Register the service as a Singleton
builder.Services.AddSingleton<IContactService, ContactService>();

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios,
see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Contact}/{action=ShowContacts}/{id?}");

app.Run();
```

OUTPUT:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7135
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5050
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-31\ContactManagementApp\ContactManagementApp
```